KKO 2025/2026 – Grayscale Image Compressor

LZSS-Based Dictionary Compression

Martin Pribylina (xpriby19)

May 2026

1 Problem Overview

The task is to write a console application in C++ that compresses 8-bit grayscale images stored in the RAW format using a lossless **data compression algorithm** based on the sliding-window principle. The program `lz_codec` has two operating modes: compression (`-c`) and decompression (`-d`). During compression the user can optionally enable a pixel-difference pre-processing step (`-m`) and an adaptive per-block scanning mode (`-a`). Everything the decompressor needs to reconstruct the original image must be embedded in the compressed file so that no extra parameters are required when decompressing.

2 Chosen Solution

Algorithm: LZSS

LZSS (Lempel–Ziv–Storer–Szymanski, 1982) is a sliding-window dictionary method. For each input position it searches a look-back buffer for the longest repetition. If a match of length $\geq$ `MIN_MATCH` is found, a *match token* `(offset, length)` is emitted; otherwise a *literal byte* is emitted. A single **flag bit** distinguishes the two cases, so short literals are not penalised with unused fields (unlike plain LZ77).

Parameters used in this implementation:

Parameter	Value	Reason
Search window	4 096 B	12-bit offset, covers one full image row and beyond
Lookahead	32 B	5-bit length field, maximum match 34 bytes
Min match	3 B	A match token (18 bits) beats three literals (27 bits)
Flag packing	8 flags/byte	Keeps the bit-stream byte-aligned

The encoder uses **hash chains** (2-byte hash, chain depth 128) to locate matches quickly. The decoder only reads and copies data, with no search at all.

Scanning and Serialization

Because images are two-dimensional, the pixels must be arranged into a one-dimensional byte sequence before compression. The order of this sequence has a direct effect on how many repeating patterns LZSS can find.

Static mode (default): pixels are scanned row by row (horizontal). The entire image is treated as one contiguous byte stream.

Adaptive mode (`-a`): the image is divided into 16×16 pixel blocks. For each block the *sum of absolute differences* (SAD) is computed along rows and along columns; the direction with the lower SAD is chosen because smoother transitions produce more repetition and longer matches. If LZSS cannot compress a block (encoded size $\geq$ raw size), the block is stored as-is. Two metadata bits per

block (scan direction and compression flag) are packed into a compact bit array written before the block data.

Pixel-Difference Model (`-m`)

Before LZSS encoding, each byte is replaced by its difference from the previous byte:

$$\delta_i = x_i - x_{i-1} \pmod{256}, \quad \delta_0 = x_0.$$

For smooth images, differences cluster near zero, creating more repeating byte sequences and therefore longer LZSS matches. The inverse transform ($x_i = x_{i-1} + \delta_i$) is applied after decoding.

File Format

The compressed file header (9 bytes, little-endian):

Offset	Size	Field
0	2 B	`uint16_t` image width
2	2 B	`uint16_t` image height
4	1 B	flags: bit 0 = model used, bit 1 = adaptive, bit 2 = raw fallback
5	4 B	`uint32_t` original pixel count

Bit 2 of flags (*raw fallback*) is set when compression would produce output larger than the input. In that case the payload is the original raw pixels and the output size equals the input size plus the 9-byte header.

3 Key Data Structures

BitWriter / BitReader (`bitio.hpp`): accumulate bits into bytes using a 1-byte buffer. `flush()` zero-pads the final byte. Reading and writing arbitrary bit widths allows compact LZSS tokens without byte-alignment waste.

Window (circular buffer, 4096 bytes): maintained identically by both encoder and decoder so the dictionary does not need to be transmitted.

HashChains: two arrays (`head[65536]`, `prev[4096]`) index the window by the 2-byte hash of each position. Walking the chain up to depth 128 finds long matches without scanning the entire window.

FlagGroup: accumulates up to 8 tokens (literals or match references), then writes one flag byte followed by the payloads in order. This keeps the bit-stream layout identical on both sides.

Block / BlockMeta: holds the 16×16 pixel sub-image, its chosen scan direction (0 = horizontal, 1 = vertical), and its compression flag. Block metadata is bit-packed (2 bits per block) into a compact array before the block payloads.

4 Results

All measurements are on 512×512 (262 144 B) 8-bit grayscale images compiled with `g++ -O2 -std=c++17` on a Linux x86-64 host. Entropy H is the first-order (symbol-independent) Shannon entropy of each image.

	Static		Static+M		Adaptive		Adaptive+M		
Image	bpp	ms	bpp	ms	bpp	ms	bpp	ms	H
cb	0.56	47	0.57	44	0.76	203	0.77	197	1.00
cb2	0.68	49	0.70	51	2.29	200	0.91	195	6.91
df1h	0.57	44	0.56	43	1.82	199	0.91	201	8.00
df1hvx	0.61	57	0.67	61	2.06	198	1.65	208	4.51
df1v	0.58	50	0.56	42	1.82	209	0.91	188	8.00
nk01	8.00	180	8.00	177	8.00	151	8.00	152	6.47
shp	0.63	48	0.62	46	2.10	212	2.20	215	0.87
shp1	1.25	66	1.29	67	2.02	210	2.10	211	1.90
shp2	1.46	79	1.54	83	1.92	205	2.01	208	1.87
avg	**1.59**	**69**	**1.61**	**68**	**2.53**	**198**	**2.16**	**197**	**4.39**

Observations:

- **Static mode is the best overall (1.59 bpp average)**, which is perhaps surprising. Adaptive mode breaks the image into small independent blocks and each block starts with an empty LZSS window, losing all the match history from previous blocks. Static mode compresses the whole image as one stream, so the sliding window accumulates repetitions across entire rows and even across rows, giving it a significant advantage on most images.
- **The pixel-difference model helps only for correlated images.** For gradients and smooth images (df1h, df1v) the model drops the static result from 0.57 to 0.56 bpp and the adaptive result from 1.82 to 0.91 bpp – a major win. For images with sharp edges and little spatial correlation (shp1, shp2) the model slightly hurts, because pixel differences there are just as random as the original values. This was expected from the theory.
- **Adaptive + Model recovers most of the adaptive penalty** for correlated images (cb2: 2.29 → 0.91, df1h: 1.82 → 0.91), since the delta transform effectively re-creates long repeating runs even in short 256-byte blocks.
- **nk01 cannot be compressed.** Its entropy is 6.47 bpp and it contains no repeating structure. All four modes produce output equal to the input (8.00 bpp includes the 9-byte header overhead) because the raw fallback is triggered. This was expected.
- **Adaptive mode is about four times slower** than static mode. This is expected because it runs two passes per block: first the SAD heuristic to decide the scan direction, then the actual LZSS encoding. Static mode is one single pass over the whole image.

References

1. Přednášky k předmětu Kódování a komprese dat, FIT VUT (2025/2026).
2. Storer, J.A., Szymanski, T.G.: *Data compression via textual substitution*, JACM, 1982.
3. Salomon, D.: *Data Compression – The Complete Reference*, Springer, 2000.